



数仓工具

1. 学习目标

能够使用Hue操作HDFS

能够使用Hue操作Hive

理解为什么选择Sqoop

理解Sqoop1和Sqoop2的区别

理解Sqoop抽取数据的两种方式

能够使用Sqoop导入完整数据到HDFS

能够使用Sqoop导入完整数据到Hive

能够使用Sqoop导入条件数据到HDFS

能够使用Sqoop导入条件数据到Hive

能够使用Sqoop导出数据到Mysql

理解为什么选择Oozie

能够使用Hue操作Oozie

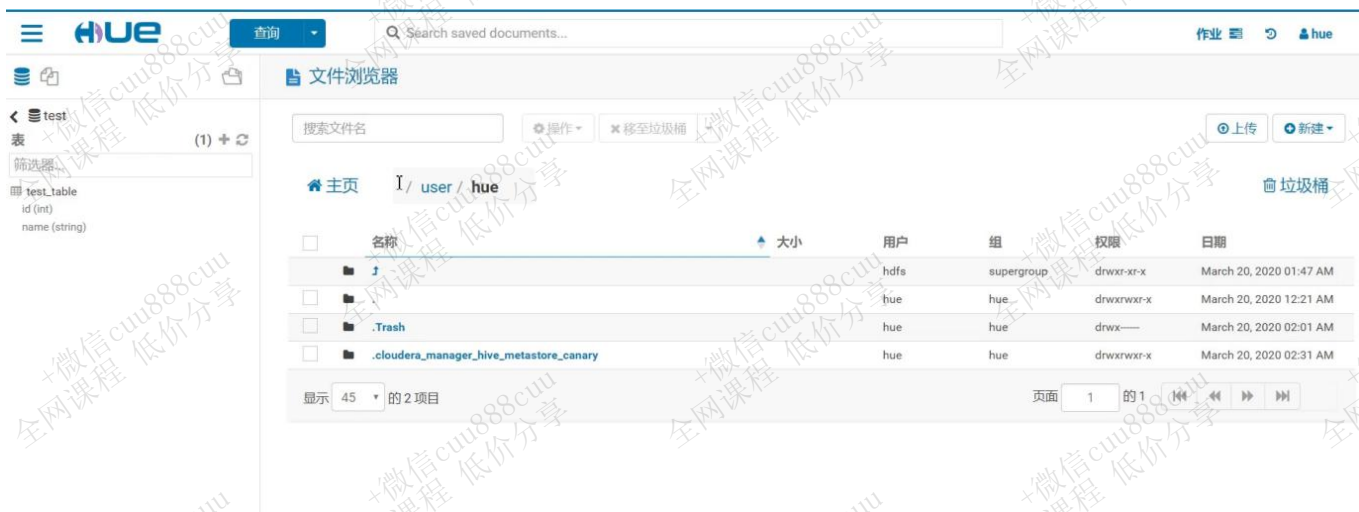




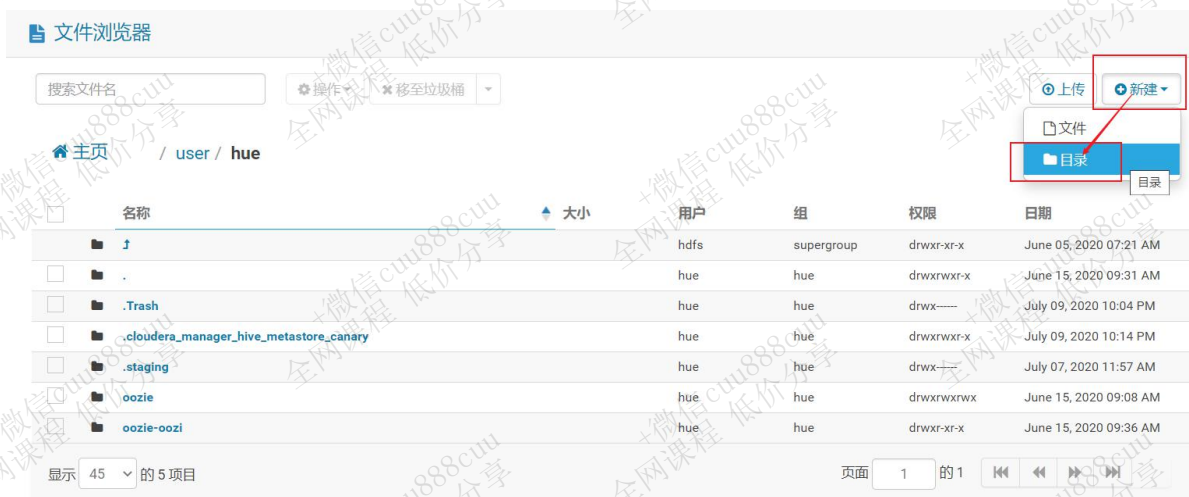
2. Hue操作HDFS

2.1 进入HDFS管理界面





2.2 HDFS新建文件夹





主页

/ user / hue

☐	名称	大小
☐	↑	
☐	.	
☐	.Trash	
☐	.cloudera_manager_hive_metastore_canary	
☐	.staging	
☐	input	
☐	oozie	
☐	oozie-oozi	
显示 45 的 6 项目		

2.3 新建文件

2.3.1 进入文件夹





主页

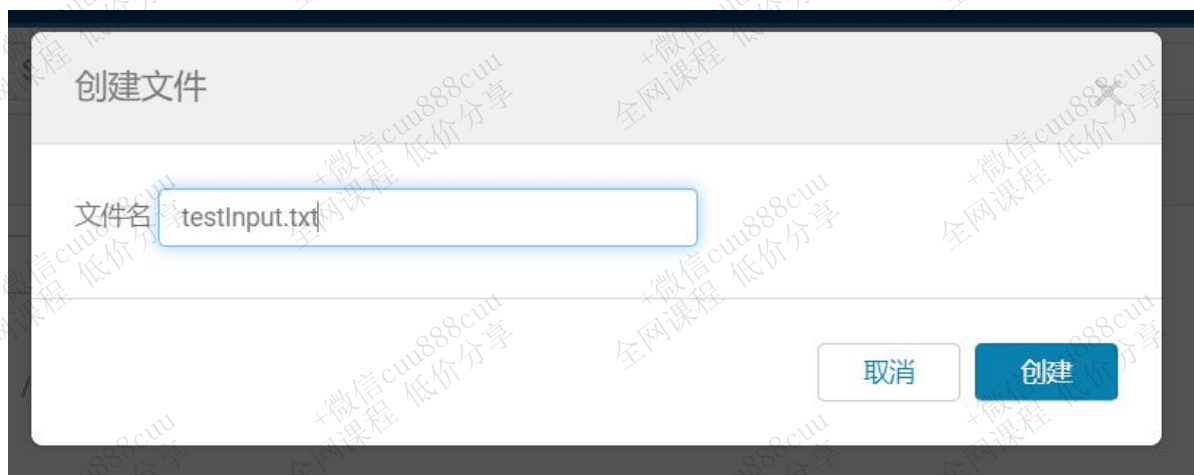
/ user / hue

	名称
☐	↑
☐	.
☐	.Trash
☐	.cloudera_manager_hive_metastore_canary
☐	.staging
☐	input
☐	oozie
☐	oozie-oozi
显示 45 的 6 项目	

点击文件夹进入

2.3.2 新建文件





2.3.3 创建成功

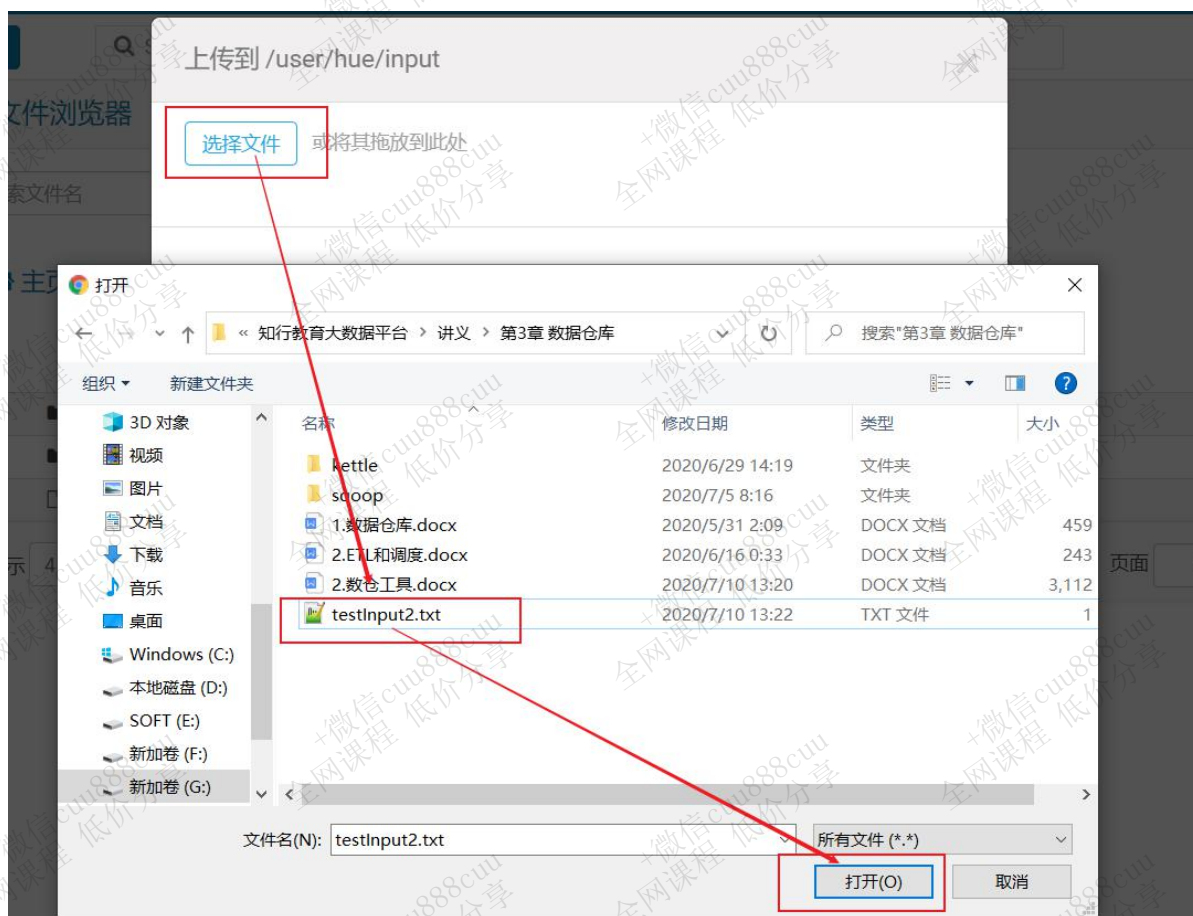




2.4 上传文件

2.4.1 选择文件





2.4.2 上传成功



2.5 查看HDFS文件内容

The screenshot shows the HDFS File Browser interface. The breadcrumb path is `/ user / hue / input / testInput2.txt`. The file content is displayed in a text area:

```
I love Beijing
I love China
Beijing is the capital of China
```

On the left sidebar, the file details are listed:

- 上次修改: 2020-07-10 下午1点23分 +08:00
- 用户: hue
- 组: hue
- 大小: 61 B
- 模式: 100644

2.6 编辑HDFS文件

The screenshot shows the HDFS File Browser interface, similar to the previous one, but with the `编辑文件` (Edit File) button in the left sidebar highlighted with a red box. The breadcrumb path is `/ user / hue / input / testInput2.txt`. The file content is displayed in a text area:

```
I love Beijing
I love China
Beijing is the capital of China
```

On the left sidebar, the file details are listed:

- 上次修改: 2020-07-10 下午1点23分 +08:00
- 用户: hue
- 组: hue
- 大小: 61 B
- 模式: 100644





文件浏览器

查看文件

主页

/ user / hue / input / testInput2.txt

上次修改

2020-07-10 下午1点23
分+08:00

用户

hue

组

hue

大小

61 B

模式

100644

```
I love Beijing  
I love China  
Beijing is the capital of China
```

Save

另存为



2.7 删除文件



2.8 更改文件权限





更改权限

	用户	组	其他
读取	☒	☒	☒
写	☒	☐	☐
执行	☐	☐	☐
易贴			☐
递归			☐

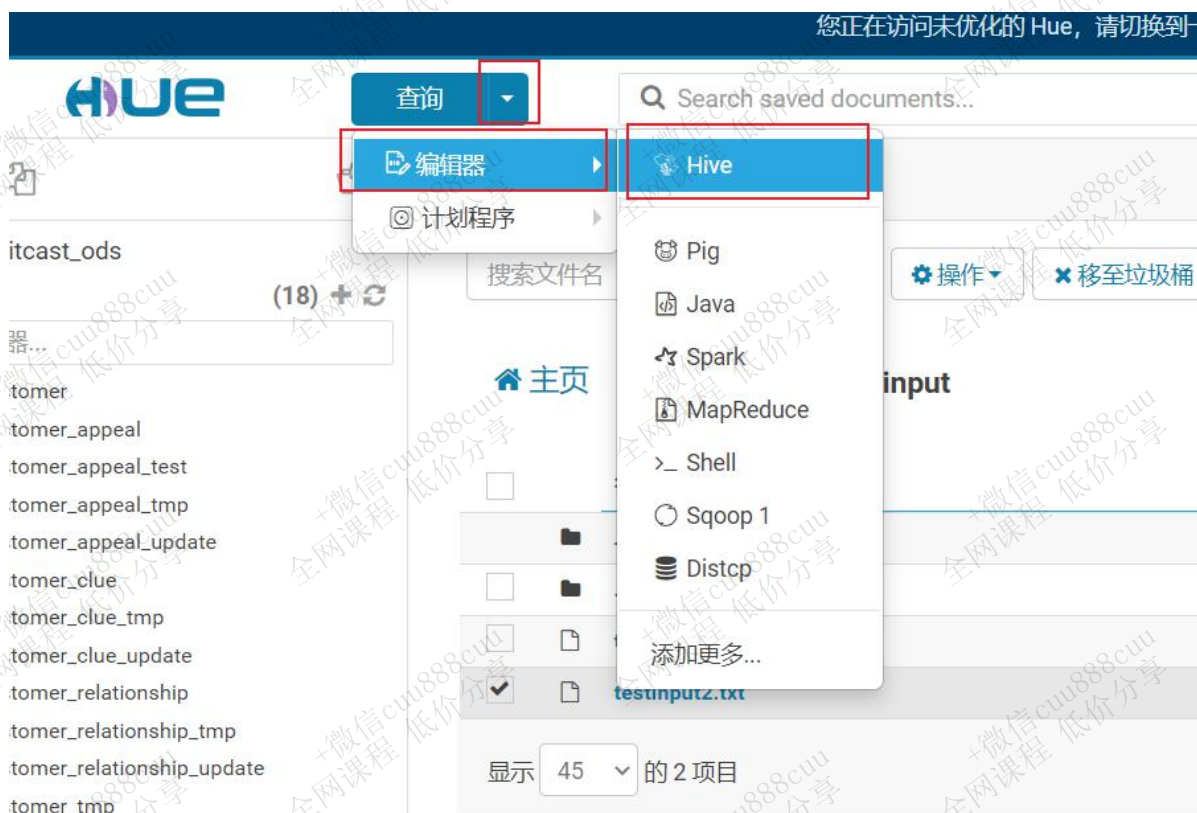
取消

提交



3. Hue操作Hive

3.1 进入Hive面板

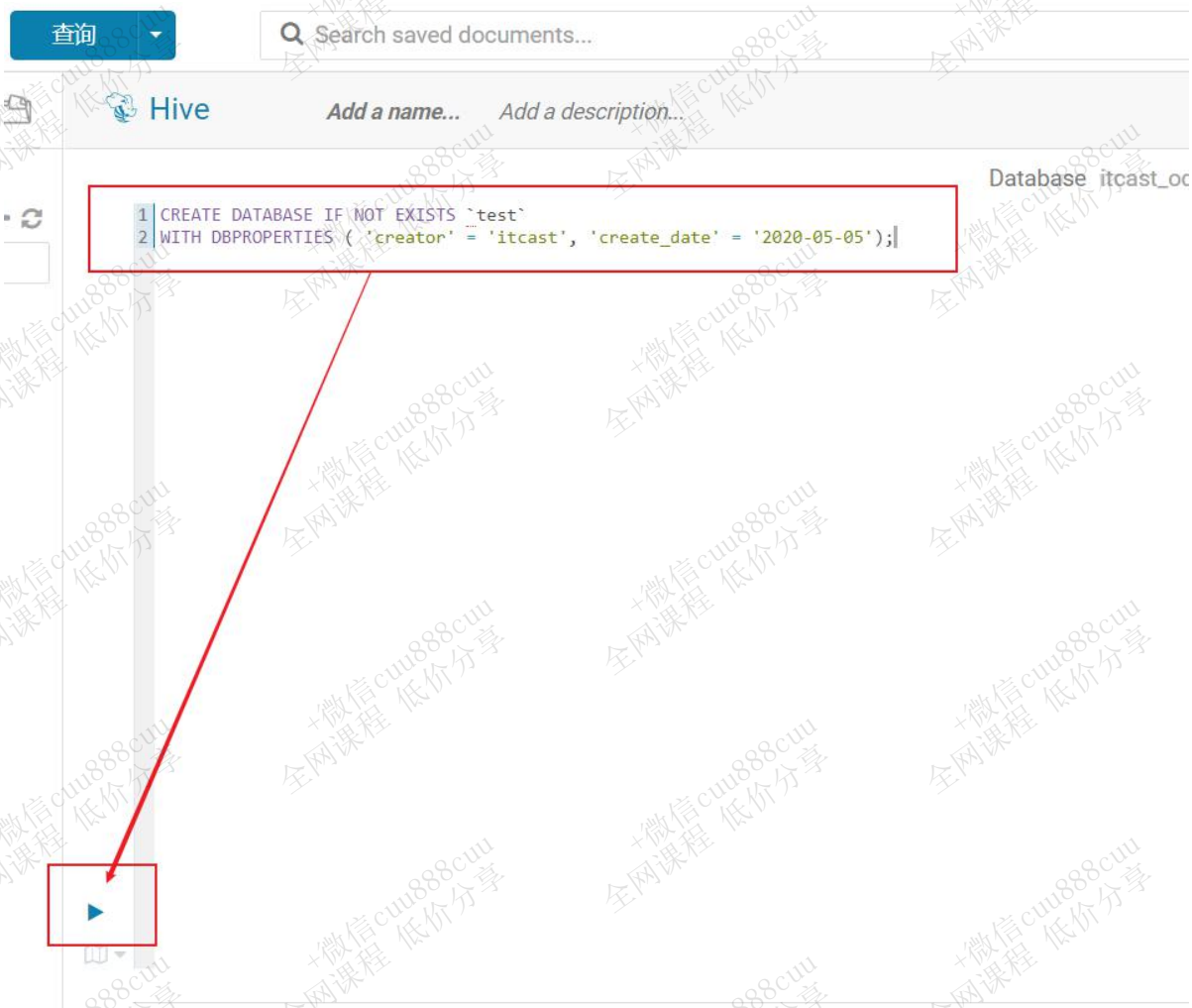


3.2 测试

3.2.1 创建数据库

```
CREATE DATABASE IF NOT EXISTS `test`;
```





3.2.2 创建表

```
create table test.test_table(  
    id int,  
    name string comment '姓名'  
)  
comment '测试表'  
row format delimited fields terminated by '\t';
```





```
1 CREATE DATABASE IF NOT EXISTS `test`
2 WITH DBPROPERTIES ( 'creator' = 'itcast', 'create_date' = '2020-05-05');
3
4 create table test.test_table(
5     id int,
6     name string comment '姓名'
7 )
8 comment '测试表'
9 row format delimited fields terminated by '\t';
```



3.2.3 插入数据

```
insert into test.test_table values (1, '张三');
```



20.50s Data

```
1 CREATE DATABASE IF NOT EXISTS `test`
2 WITH DBPROPERTIES ( 'creator' = 'itcast', 'create_date' = '2020-05-05');
3
4 create table test.test_table(
5     id int,
6     name string comment '姓名'
7 )
8 comment '测试表'
9 row format delimited fields terminated by '\t';
10
11 insert into test.test_table values (1, '张三');
```

3.2.4 查询数据

```
select * from test.test_table;
```




```
1 CREATE DATABASE IF NOT EXISTS `test`
2 WITH DBPROPERTIES ( 'creator' = 'itcast', 'create_date' = '2020-05-05');
3
4 create table test.test_table(
5     id int,
6     name string comment '姓名'
7 )
8 comment '测试表'
9 row format delimited fields terminated by '\t';
10
11 insert into test.test_table values (1, '张三');
12
13 select * from test.test_table;
```

INFO : Compiling command(queryId=hive_20200710134317_7af6ae70-5f28-4cee-8a53-2c1322ba5d08):

select * from test.test_table

INFO : Semantic Analysis Completed

INFO : Returning Hive schema: Schema(fieldSchemas:[FieldSchema(name=test_table.id, type=int, comment=null), FieldSchema(name=test_table.name, type:string, comment=null)], properties:null)

查询历史记录

保存的查询

结果 (1)

test_table.id

test_table.name

1

1

张三

Hive 查询界面截图。左侧显示表列表，中间显示查询语句，右侧显示查询结果。

查询语句：

```
1 create table test.test_table(
2     id int,
3     name string comment '姓名'
4 )
5 comment '测试用表'
6 row format delimited fields terminated by '\t';
```

查询结果：

test_table.id	test_table.name
1	张三

查询历史记录：

时间	操作
几秒前	create table test.test_table(id int, name string comment '姓名') comment '测试用表' row format delimited fields terminated by '\t'
6分钟前	create database test



3.2.5 调整区域大小

The screenshot shows the Hive CLI interface. At the top, there's a header with 'hive' and buttons for 'Add a name...' and 'Add a description...'. Below this, the SQL command is entered in a text area:

```
1 CREATE DATABASE IF NOT EXISTS `test`
2 WITH DBPROPERTIES ( 'creator' = 'itcast', 'create_date' = '2020-05-05');
3
4 create table test.test_table(
5   id int,
6   name string comment '姓名'
7 )
8 comment '测试表'
9 row format delimited fields terminated by '\t';
10
11 insert into test.test_table values (1, '张三');
12
13 select * from test.test_table;
```

Below the SQL command, the execution status is shown: '0.18s Database test 类型 text'. The output of the command is displayed in a scrollable area:

```
INFO : Compiling command(queryId=hive_20200710134317_7af6ae70-5f28-4cee-8a53-2c1322ba5d08):
select * from test.test_table
INFO : Semantic Analysis Completed
INFO : Returning Hive schema: Schema(fieldSchemas:[FieldSchema(name=test_table.id, type=int, comment:null), FieldSchema(name=test_table.name, type:string, comment:null)], properties:null)
```

Two red boxes with arrows point to specific areas in the output:

- One box points to the 'INFO : Compiling command...' line, with the text '鼠标拖动此处改变sql区域大小' (Drag the mouse here to change the SQL region size).
- Another box points to the 'INFO : Returning Hive schema...' line, with the text '鼠标拖动此处，改变执行过程区域大小' (Drag the mouse here, change the execution process region size).

At the bottom, the results are shown in a table:

test_table.id	test_table.name
1	张三

3.2.6 Hive内置函数

在SQL中提供了很多的内置函数，或者叫内嵌函数，包括聚合函数、数学函数，字符串函数、转换函数，日期函数，条件函数，表生成函数等等。

3.2.6.1 数学函数

指定精度取整函数: round

语法: round(double a, int d)

说明: 返回指定精度d的double类型

举例: hive> select round(3.1415926,4);

3.1416



3.2.6.2 字符串函数

3.2.6.2.1 字符串截取函数

大多数数据库中都有substr和substring两种字符串截取函数。但与其他的数据库不同，在hive中，**substr与substring函数的使用方式是完全一致的**，属于同一个函数。

3.2.6.2.1.1 两个参数

语法：substr(string A, int start), substring(string A, int start)

返回值: string

说明：返回字符串A从start位置到结尾的字符串

栗子：

```
select substr('2020-06-06', 6), substring('2020-06-06', 6);    --06-06  06-06
```

3.2.6.2.1.2 三个参数

语法：substr(string A, int start, int len), substring(string A, intstart, int len)

返回值: string

说明：返回字符串A从start位置开始，**长度**为len的字符串。

栗子：

```
select substr('2020-06-06', 6, 2), substring('2020-06-06', 6,2);    --06  06
```

3.2.6.2.2 字符串拼接函数

3.2.6.2.2.1 concat

语法：concat(string A, string B...)

返回值: string

说明：返回输入字符串连接后的结果，支持任意个输入字符串

```
hive> select concat('hello','world');
```

```
helloworld
```





3.2.6.2.2 concat_ws

语法: concat_ws(string SEP, string A, string B...)

返回值: string

说明: 返回输入字符串连接后的结果，SEP表示各个字符串间的分隔符

举例:

```
hive> select concat_ws(',', 'aaa', 'bbb', 'ccc');  
aaa,bbb,ccc
```

3.2.6.3 日期函数

3.2.6.3.1 year/quarter/month/day/hour

语法:

year(string date)

month(string date)

day(string date)

hour(string date)

说明: 返回日期中的年/月/日/小时。

举例:

```
hive> select year('2012-12-08');  
2012  
hive> select quarter('2012-12-08');  
12  
hive> select month('2012-12-08');  
8  
hive> select hour('2012-12-08 13:35:35');  
13
```



3.2.6.3.2 日期格式化函数

date_format(string time, string format)

说明:返回指定格式的日期

举例:

```
select date_format('2020-1-1', 'yyyy-MM-dd'); -- 2020-01-01
```

```
select date_format('2020-1-1 12:23', 'yyyy-MM-dd'); -- 2020-01-01
```

```
select date_format('2020-1-1 12:23:35', 'yyyy-MM-dd'); -- 2020-01-01
```

```
select date_format('2020-1-1 1:1:1', 'yyyy-MM-dd HH:mm:ss');--2020-01-01 01:01:01
```

```
select hour(date_format('2020-1-1 18:1:00', 'yyyy-MM-dd HH:mm:ss')); --18
```

3.2.6.4 判断函数

3.2.6.4.1 If函数

if和case差不多，都是处理单个列的查询结果

语法: if(boolean testCondition, T valueTrue, T valueFalseOrNull)

返回值: T

说明: 当条件testCondition为TRUE时，返回valueTrue；否则返回valueFalseOrNull。

示例: if (条件表达式, 结果1, 结果2) 相当于java中的三目运算符,只是if后面的表达式类型可以不一样。

```
select if(a='a','bbbb',111) from lxw_dual;
```

```
bbbb
```

```
select if(1<2,100,200) from lxw_dual;
```

```
200
```

3.2.6.4.2 CASE WHEN THEN

语法: CASE WHEN a THEN b [WHEN c THEN d]* [ELSE e] END



说明：如果a为TRUE,则返回b；如果c为TRUE，则返回d；否则返回e

如：CASE WHEN 5>0 THEN 5 WHEN 4>0 THEN 4 ELSE 0 END 将返回5；CASE WHEN 5<0 THEN 5 WHEN 4<0 THEN 4 ELSE 0 END 将返回0。

示例：

```
select case when 1=2 then 'tom' when 2=2 then 'mary' else 'tim' end from lxw_dual;
mary
select case when 1=1 then 'tom' when 2=2 then 'mary' else 'tim' end from lxw_dual;
tom
```

4. Sqoop

4.1 Sqoop介绍

Sqoop是Apache下的顶级项目，用来将Hadoop和关系型数据库中的数据相互转移，可以将一个关系型数据库（例如：MySQL，Oracle，PostgreSQL等）中的数据导入到Hadoop的HDFS中，也可以将HDFS的数据导入到关系型数据库中。目前在各个公司应用广泛，且发展前景比较乐观。其特点在于：

- 1) 专门为Hadoop而生，随Hadoop版本更新支持程度好，且原本即是从CDH版本孵化出来的开源项目，支持CDH的各个版本号。
- 2) 它支持多种关系型数据库，比如mysql、oracle、postgresql等。
- 3) 可以高效、可控的利用资源。
- 4) 可以自动的完成数据映射和转换。
- 5) 大部分企业还在使用sqoop1版本，sqoop1能满足公司的基本需求。
- 6) 自带的辅助工具比较丰富，如sqoop-import、sqoop-list-databases、sqoop-list-tables等。

4.2 为什么选择Sqoop

我们常用的ETL工具有Sqoop、Kettle、Nifi。

知行教育大数据平台，ETL的数据量较大，但是数据来源的类型简单（mysql）：

1. Kettle虽然功能较完善，但当处理大数据量的时候瓶颈问题比较突出，不适合此项目；
2. NiFi的功能强大，且支持大数据量操作，但NiFi集群是独立于Hadoop集群的，需要独立的





服务器来支撑，强大也就意味着有上手门槛，**学习难度大，用人成本高**；

3. Sqoop专为**关系型数据库**和**Hadoop**之间的ETL而生，支持**海量数据**，符合项目的需求，且操作简单**门槛低**，因此选择Sqoop作为ETL工具。

4.3 Sqoop操作

Sqoop详细资料见 《Home\讲义\第2章 数据仓库与工具\sqoop\sqoop教程_V1.0.docx》。

